



NATURAL LANGUAGE PROCESSING

Project Title

SMS SPAM DETECTION USING NATURAL LANGUAGE PROCESSING AND MACHINE LEARNING

Programme	BBA (Artificial Intelligence)
Academic Year	2026-27
Team Members	1. Sarvesh Wani - 25030422140 2. Sanando Maitra - 25030422105 3. Priyanshu Akshat - 25030422095
Faculty	Dr. Nabanita Choudhury, Assistant Professor

SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE (SAII)

Constituent of Symbiosis International (Deemed University)

Lavale, Pune - 412115

DECLARATION

We hereby declare that the Project Report entitled "**SMS Spam Detection Using Natural Language Processing and Machine Learning**", which is being submitted to **Symbiosis Artificial Intelligence Institute (SAII), Symbiosis International (Deemed University)**, in partial fulfilment of the requirements for the completion of the third semester of the **Bachelor of Business Administration (Artificial Intelligence)**, is a bonafide report of the work carried out by us. The material contained in this report has not been submitted to any other university or institution.

Place: SAI, SIU, Pune

Date: 10/10/2026

Sarvesh Wani (PRN: 25030422140)

Sanando Maitra (PRN: 25030422105)

Priyanshu Akshat (PRN: 25030422095)

CERTIFICATE

This is to certify that the Project Report entitled "**SMS Spam Detection Using Natural Language Processing and Machine Learning**", submitted by Mr. Sarvesh Wani (PRN: 25030422140), Mr. Sanando Maitra (PRN: 25030422105) and Mr. Priyanshu Akshat (PRN: 25030422095) as a record of the work carried out by them, is accepted as a report submission in partial fulfilment of the requirements for the completion of the third semester of the **Bachelor of Business Administration (Artificial Intelligence)**.

Project Guide

Dr. Nabanita Choudhury

Assistant Professor, Course Instructor

Symbiosis Artificial Intelligence Institute

TABLE OF CONTENTS

1. Abstract.....	7
2. Introduction.....	8
3. Problem Statement and Objectives	9
4. Dataset Description	10
5. Data Preprocessing	14
6. Feature Representation / Embeddings	16
7. Proposed Methodology / System Architecture	18
8. NLP Model / Technique Used	20
9. Implementation	21
10. Results and Evaluation	23
11. Error Analysis / NLP Failure Analysis.....	27
12. Ethical Considerations and Limitations.....	29
13. Conclusion	30
14. Future Scope	30
15. References.....	32
16. GitHub Repository	32
17. Team Contribution	33
18. Annexure.....	34

LIST OF TABLES

Table 1. Dataset structure used in the project.....	10
Table 2. Dataset cleaning summary.	11
Table 3. Class distribution after deduplication.	11
Table 4. Average message length by class.	12
Table 5. Examples of the preprocessing output.	15
Table 6. Train-test and TF-IDF configuration.	16
Table 7. Machine learning models and their settings (random_state=42 for all models that use it).	20
Table 8. Programming language and key libraries.	21
Table 9. Predictions on manually supplied new messages.....	22
Table 10. Model performance comparison on the test set (spam as the positive class).....	23
Table 11. Detailed classification report for Linear SVM.	25
Table 12. Error counts from the confusion matrices and derived error rates (test set: 903 ham, 131 spam).	27
Table 13. Failure cases visible in the preprocessing and prediction outputs.....	27
Table 14. Contribution of each team member.	33

LIST OF FIGURES

Figure 1. Class distribution of ham and spam messages.	11
Figure 2. Message length distributions (characters and words) by class.	12
Figure 3. Top 20 words in spam and ham messages after cleaning.	13
Figure 4. Spam and ham word clouds.	13
Figure 5. End-to-end workflow of the SMS spam detection system.	18
Figure 6. Output of the new-message test in the implementation.	22
Figure 7. Model comparison across accuracy, precision, recall, F1-score and ROC-AUC.	23
Figure 8. Confusion matrices for the four classifiers (Naive Bayes, Logistic Regression, Linear SVM, Random Forest).	24
Figure 9. Precision-Recall curves for model comparison.	25
Figure 10. Most influential Logistic Regression features for spam and ham.	26

1. Abstract

SMS spam is a common text-classification problem where unwanted promotional or fraudulent messages need to be separated from genuine messages (ham). In this project, we built a complete Natural Language Processing pipeline for detecting SMS spam using the UCI SMS Spam Collection. The dataset contains 5,572 real UK English messages. After checking the data and removing 403 duplicate messages, 5,169 unique messages remained, including 4,516 ham and 653 spam messages, giving an imbalance of about 6.9:1.

The messages were first cleaned by converting them to lowercase, normalizing URLs, currency symbols and numbers, removing punctuation, tokenizing the text, removing stopwords and applying lemmatization. The cleaned text was then converted into TF-IDF features using unigrams and bigrams, with 3,000 features retained. We trained and compared four classifiers: Multinomial Naive Bayes, Logistic Regression, Linear SVM and Random Forest. Class weighting was used for the models that support it. The models were evaluated on a stratified test set of 1,034 messages using accuracy along with precision, recall, F1-score and ROC-AUC.

Linear SVM produced the strongest overall result, achieving 98.26% accuracy, 94.49% precision, 91.60% recall, 93.02% F1-score and 0.9939 ROC-AUC. The final pipeline also classified all three manually written test messages correctly. Overall, the project shows that a traditional NLP pipeline using TF-IDF and machine learning can work very well for SMS spam detection. It also showed us why careful data splitting, handling class imbalance and looking at more than one evaluation metric are important.

2. Introduction

2.1 Background and Context

SMS is still a simple and familiar way of communicating, but it is also commonly used for unwanted advertisements, fraudulent offers, prize messages and other misleading content. Because of this, automatically identifying suspicious messages is a useful binary text-classification task. Each message is classified as either legitimate (ham) or spam.

2.2 Motivation for Choosing the Problem

Checking a large number of messages manually would take a lot of time and would not be practical. An automated classifier can act as a quick first-level filter. We chose this problem because it also gives a clear example of a practical NLP workflow: raw text has to be cleaned and converted into a useful representation before a machine learning model can make predictions. The UCI dataset is also a well-known academic benchmark, making it easier to understand and compare the results.

2.3 Role of NLP in the Proposed Solution

NLP helps convert unstructured language into a form that machine learning algorithms can work with. In our project, the raw SMS messages are gradually cleaned through lowercasing, normalization, tokenization, stopword removal and lemmatization. They are then converted into numerical TF-IDF features. The machine learning models use these features to learn patterns that help separate ham messages from spam.

2.4 Scope and Applications

The project focuses on the binary classification pipeline implemented in the submitted Python/Google Colab project. It includes data inspection, exploratory analysis, text preprocessing, TF-IDF vectorization, traditional machine learning, model evaluation, interpretation and testing with new messages. Transformer models and deep learning are outside the scope of this implementation. The approach could be used for:

- Message filtering and spam screening.
- Communication platforms that need an initial spam classification layer.
- Fraud-awareness systems that flag suspicious promotional messages.
- Academic demonstrations of NLP text classification.
- Foundational components for larger messaging-security systems.

3. Problem Statement and Objectives

3.1 Problem Statement

The main aim of the project is to automatically classify SMS messages as ham or spam. To do this reliably, the system needs to handle informal language, punctuation, URLs, currency symbols, numbers and common words, while also dealing with the imbalance between the two classes. The train-test separation must also be maintained carefully so that the evaluation remains reliable.

3.2 Objectives

1. To inspect and prepare the UCI SMS Spam Collection dataset.
2. To remove duplicate messages before model development.
3. To explore the distribution and characteristics of ham and spam messages.
4. To implement a text-cleaning and preprocessing pipeline.
5. To represent messages using TF-IDF unigrams and bigrams.
6. To train and compare Multinomial Naive Bayes, Logistic Regression, Linear SVM, and Random Forest classifiers.
7. To evaluate the models using accuracy, precision, recall, F1-score, ROC-AUC, confusion matrices, and related curves.
8. To test the final pipeline on new SMS messages.

4. Dataset Description

4.1 Dataset Name, Source and Overview

We used the SMSSpamCollection dataset, also known as the UCI SMS Spam Collection, from the UCI Machine Learning Repository (<https://archive.ics.uci.edu/dataset/228/sms+spam+collection>). It contains 5,572 real SMS messages written in UK English. Each record has a label showing whether the message is ham or spam and the original message text. The source file is tab-separated and does not contain a header row.

The messages cover a fairly broad range of vocabulary rather than focusing on one specific type of SMS. This is useful because a model trained on varied ham and spam examples has a better chance of handling wording that it has not seen before than a model trained on a very narrow set of messages.

4.2 Dataset Structure

Table 1. Dataset structure used in the project.

Column	Description	Type
label	Original class label: ham or spam	Object
message	Raw SMS message text	Object
spam	Binary representation: 1 for spam and 0 for ham	Integer

4.3 Data Loading and Inspection

We load the dataset with pandas using a tab separator. Since the original file has no header, we assign the columns the names label and message. We then create a third column, spam, by converting the original text label into a binary value.

```
df = pd.read_csv('SMSSpamCollection', sep='\t', header=None,
                 names=['label', 'message'])
df['spam'] = (df['label'] == 'spam').astype(int)
```

The initial DataFrame contains 5,572 rows and three columns. The inspection results show 5,572 non-null values in each column, so no missing values were found in label, message or spam.

4.4 Duplicate Removal

The implementation finds 403 duplicate rows and 403 duplicate messages. We remove duplicates based on the message field before performing the train-test split. This is important because if identical messages appeared in both sets, the model could appear to perform better than it actually does on unseen data.

Table 2. Dataset cleaning summary.

Stage	Rows / Records
Initial dataset	5,572
Duplicate messages identified	403
Dataset after deduplication	5,169
Missing values	0

4.5 Class Distribution

After removing duplicates, 5,169 unique messages remain. There are 4,516 ham messages and 653 spam messages, which means that about 87.4% of the data is ham and 12.6% is spam. The resulting imbalance is approximately 6.9:1.

Table 3. Class distribution after deduplication.

Class	Count	Proportion
Ham (0)	4,516	87.4%
Spam (1)	653	12.6%
Total	5,169	100%

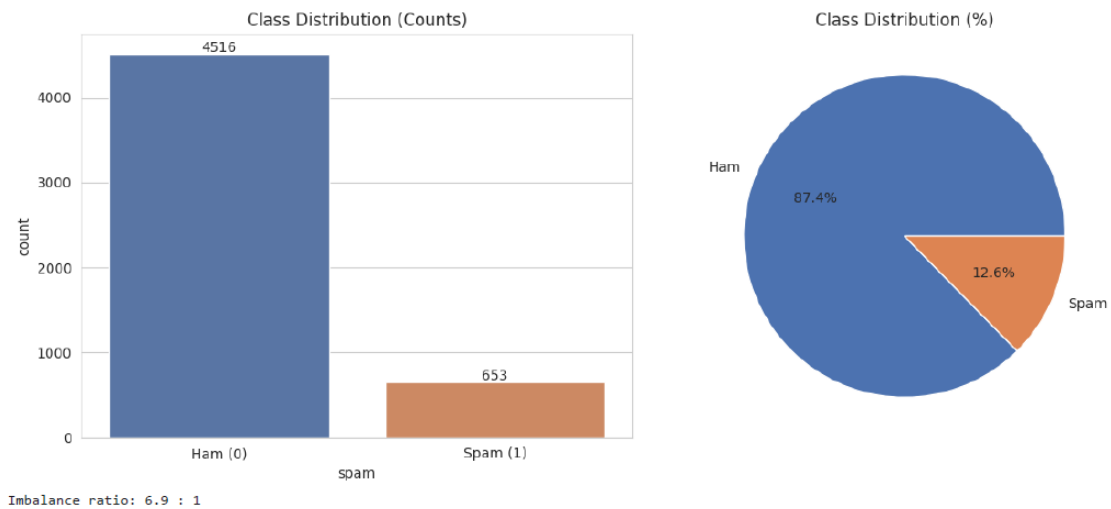


Figure 1. Class distribution of ham and spam messages.

The class-distribution chart makes the imbalance easy to see: legitimate messages form most of the dataset, while spam is the smaller class. This matters because a model could achieve high accuracy simply by predicting ham most of the time while still missing many spam messages. To reduce this problem, we use `class_weight='balanced'` for Logistic Regression, Linear SVM and Random Forest, while keeping Multinomial Naive Bayes as an unweighted baseline. We therefore pay particular attention to precision, recall, F1-score and ROC-AUC instead of relying on accuracy alone.

4.6 Message Length Analysis

We calculate both the number of characters and the number of words in each message. The averages show a noticeable difference between the two classes. Ham messages contain about 70.91 characters and 14.24 words on average, while spam messages contain about 137.70 characters and 23.74 words.

Table 4. Average message length by class.

Class	Average characters	Average words
Ham	70.905890	14.239814
Spam	137.704441	23.739663

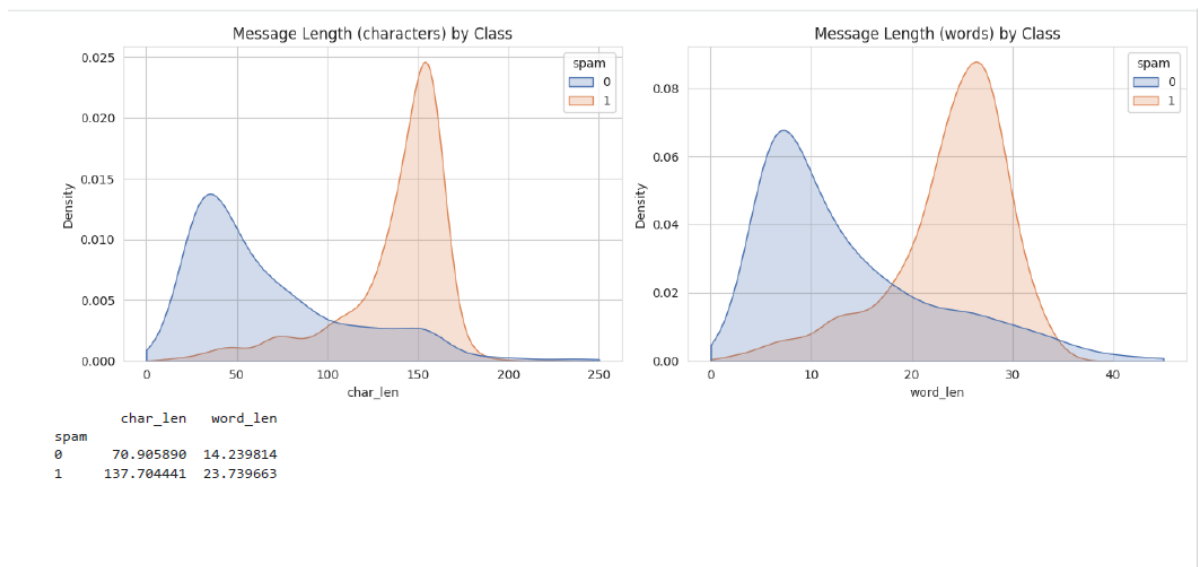


Figure 2. Message length distributions (characters and words) by class.

The density plots show that spam messages generally fall into a longer-message range than ham messages. One likely reason is that spam often includes an offer, a call to action or contact details, whereas ham messages are more often short and conversational.

4.7 Frequent Words

After cleaning the messages, we compare the 20 most frequent words in the two classes. Spam messages contain more words related to offers, prizes, calls, claims and urgency, while the ham messages are dominated by everyday conversational terms.

5. Data Preprocessing

Before extracting features, we clean the raw messages so that they have a more consistent form. The implementation performs these operations through one function, `clean_text()`, which was written specifically for this SMS dataset. The main preprocessing steps and their purpose are explained below.

5.1 Lowercasing

We convert every message to lowercase. This prevents the model from treating different capitalization of the same word, such as FREE and free, as separate forms.

5.2 URL, Currency and Number Normalization

URLs starting with `http` or `www` are replaced with the placeholder URL, while the pound and dollar symbols are replaced with CURRENCY. Numeric sequences containing four or more digits are replaced with NUM. The idea is to keep some indication that a message contains a link, an amount or a phone number/shortcode, even when the exact value is not important.

5.3 Punctuation and Non-Alphabetic Characters

Characters other than lowercase letters and whitespace are removed. We also reduce repeated spaces so that the resulting text follows a consistent format.

5.4 Tokenization

The cleaned text is divided into individual word tokens using NLTK's `word_tokenize` function. This step is needed because the later preprocessing operations work with individual words.

5.5 Stop-word Removal

Common English stopwords are removed using NLTK's stopword list. We also remove one-character tokens. These very common words generally provide little useful information for deciding whether a message is spam or ham.

5.6 Lemmatization

We use `WordNetLemmatizer` to bring words closer to their base forms. For example, receiving can be reduced toward receive. This helps group related word forms together and reduces unnecessary variation in the text.

5.7 Cleaning Function

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+|www\S+', ' URL ', text)
    text = re.sub(r'£|\$', ' CURRENCY ', text)
    text = re.sub(r'\b\d{4,}\b', ' NUM ', text)
    text = re.sub(r'^a-z\s', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t) for t in tokens
              if t not in stop_words and len(t) > 1]
    return ' '.join(tokens)
```

5.8 Examples of Cleaned Messages

Table 5. *Examples of the preprocessing output.*

Original message (shortened)	Cleaned message (shown in output)
Go until jurong point, crazy.. Available only ...	go jurong point crazy available bugis great wo...
Ok lar... Joking wif u oni...	ok lar joking wif oni
Free entry in 2 a wkly comp to win FA Cup fina...	free entry wkly comp win fa cup final tkts st ...
U dun say so early hor... U c already then say...	dun say early hor already say

After the cleaning function is applied, we check whether any messages have been left with no words. Twelve messages become empty after preprocessing. This check is useful because aggressive cleaning can sometimes remove all meaningful tokens from a very short message.

There is also an important issue with the placeholder tokens. URL, CURRENCY and NUM are initially written in uppercase, but the following cleaning step keeps only lowercase letters, so these placeholders are removed before tokenization. The outputs in Section 9 show that values such as £100,000 and 81010 do not remain in the cleaned text. Similarly, a link such as bit.ly/claim123 does not match the current URL pattern, which only looks for http or www, and is reduced to words such as “bit ly claim”. This issue is discussed further in Section 11.

There is also an important issue with the placeholder tokens. URL, CURRENCY and NUM are initially written in uppercase, but the following cleaning step keeps only lowercase letters, so these placeholders are removed before tokenization. The outputs in Section 9 show that values such as £100,000 and 81010 do not remain in the cleaned text. Similarly, a link such as bit.ly/claim123 does not match the current URL pattern, which only looks for http or www, and is reduced to words such as “bit ly claim”. This issue is discussed further in Section 11.

6. Feature Representation / Embeddings

6.1 Need for Numerical Representation

Machine learning classifiers cannot work directly with raw text strings, so the cleaned messages need to be converted into numerical vectors. For this project, we use Term Frequency-Inverse Document Frequency (TF-IDF), a commonly used representation for text classification.

6.2 TF-IDF and the Reason for Choosing It

TF-IDF gives more weight to words that are useful in a particular message but are not common throughout the whole collection. This fits the spam-detection task because words such as claim, prize and txt can help distinguish spam, while very common everyday words should have less influence. TF-IDF also creates sparse, high-dimensional vectors, which work well with linear models such as Logistic Regression and Linear SVM. Another advantage is that the feature weights can be inspected later to understand which words influence the predictions.

6.3 Unigrams and Bigrams

We set `ngram_range=(1,2)`, so the model considers both individual words (unigrams) and two-word combinations (bigrams). Including bigrams allows the representation to capture short phrases that may be more informative than a single word on its own.

6.4 Train-Test Split

The cleaned data is split into training and testing sets in an 80:20 ratio. We use stratification to keep the class proportions similar in both sets, and `random_state` is fixed at 42 so that the split can be reproduced. This gives 4,135 training messages and 1,034 test messages, including 903 ham and 131 spam messages in the test set.

6.5 TF-IDF Configuration

Table 6. Train-test and TF-IDF configuration.

Parameter	Value	Purpose
test_size	0.20	20% of the data is used for testing
stratify	spam labels	Maintains class proportions
random_state	42	Reproducible split
max_features	3000	Keeps the top 3,000 features
ngram_range	(1,2)	Uses unigrams and bigrams
min_df	2	Keeps terms appearing in at least two documents

```
tfidf = TfidfVectorizer(max_features=3000, ngram_range=(1,2), min_df=2)
X_train = tfidf.fit_transform(X_train_text) # fitted on training text only
X_test = tfidf.transform(X_test_text)
```


The training matrix contains 4,135 samples and 3,000 features, while the test matrix contains 1,034 samples with the same 3,000 feature dimensions. A key point is that TF-IDF is fitted only on the training text and then applied to the test text. This keeps information from the test vocabulary from leaking into model training.

7. Proposed Methodology / System Architecture

7.1 End-to-End Workflow

The final workflow moves from raw SMS text through cleaning and normalization, tokenization, stopwords removal, lemmatization, TF-IDF feature extraction and model training, followed by evaluation and prediction on new messages. This gives us a clear and reproducible path from unstructured SMS text to a binary classifier. Figure 5 summarizes the input, processing, model and output stages.

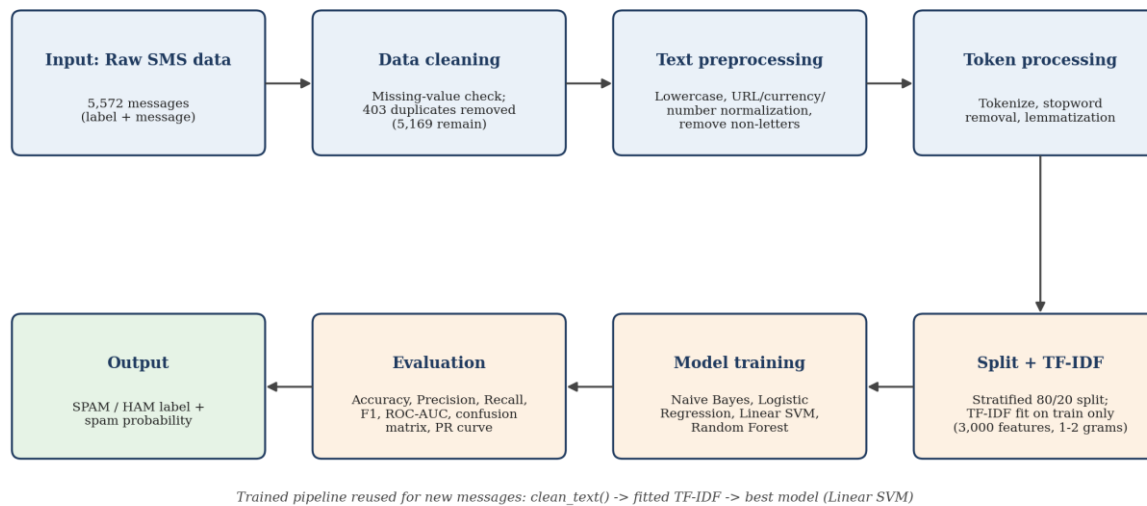


Figure 5. End-to-end workflow of the SMS spam detection system.

7.2 Stages of the Pipeline

1. **Data loading and inspection:** read the tab-separated file, assign the column names and create the binary spam label.
2. **Data cleaning:** check for missing values and remove the 403 duplicate messages.
3. **Exploratory analysis:** examine class balance, message length and frequently used words.
4. **Text preprocessing:** convert to lowercase, normalize selected values, remove punctuation, tokenize, remove stopwords and lemmatize the text.
5. **Split and vectorization:** perform a stratified 80:20 split and fit TF-IDF only on the training text.
6. **Model training:** train the four classifiers using the same training matrix.
7. **Evaluation:** compare metrics, confusion matrices, ROC curves and precision-recall curves on the held-out test set.
8. **Interpretation and testing:** examine influential features and use the pipeline to predict new messages.

7.3 Key Design Decisions

- **Leakage prevention:** duplicates are removed before the train-test split, and the TF-IDF vocabulary is learned only from the training text.
- **Imbalance handling:** `class_weight='balanced'` is used for the models that support it, and the evaluation gives more attention to precision, recall, F1-score and ROC-AUC.
- **Fair comparison:** every model receives the same TF-IDF training matrix, so the differences in performance mainly reflect the algorithms rather than different feature sets.

8. NLP Model / Technique Used

We compare four traditional machine learning algorithms. Instead of relying on a single classifier, the project tests several approaches to see which one performs best with the prepared TF-IDF features.

8.1 Multinomial Naive Bayes

Multinomial Naive Bayes is used as a standard baseline for text classification. It works naturally with features based on word occurrence or frequency. In this project, it remains an unweighted baseline because the estimator does not support class weighting in the same way as the other selected models.

8.2 Logistic Regression

Logistic Regression is a linear classifier that estimates the probability of a binary outcome. It is also useful for interpretation because its learned coefficients can show which features push a prediction toward spam or toward ham.

8.3 Linear Support Vector Machine

Linear SVM is well suited to high-dimensional, sparse text features and is a strong classical choice for this type of problem. We use LinearSVC with balanced class weights and wrap it with CalibratedClassifierCV so that probability estimates are available for evaluation and example predictions.

8.4 Random Forest

Random Forest is an ensemble model made up of multiple decision trees. We include it as a non-linear alternative so that it can be compared with the linear approaches used for text classification.

8.5 Model Configuration

Table 7. Machine learning models and their settings (*random_state=42 for all models that use it*).

Model	Configuration / Role
Multinomial Naive Bayes	Unweighted classical text-classification baseline
Logistic Regression	max_iter=1000; class_weight='balanced'
Linear SVM	LinearSVC with class_weight='balanced', calibrated probabilities
Random Forest	200 trees; class_weight='balanced'; n_jobs=-1

8.6 Training Procedure

Each classifier is trained using `X_train` and `y_train`. We then generate predictions for `X_test` and keep probability estimates for ROC-AUC and Precision-Recall analysis. The fitted

models, predictions and probabilities are stored in dictionaries so that all four algorithms can be evaluated in the same way.

9. Implementation

9.1 Programming Language and Libraries

Table 8. *Programming language and key libraries.*

Purpose	Tools
Language and environment	Python; Google Colab notebook
Data handling	pandas, NumPy, re, collections.Counter
Visualization	Matplotlib, Seaborn, WordCloud
NLP	NLTK (stopwords, punkt tokenizer, WordNet lemmatizer)
Machine learning and metrics	scikit-learn (TfidfVectorizer, MultinomialNB, LogisticRegression, LinearSVC, CalibratedClassifierCV, RandomForestClassifier, evaluation metrics)

9.2 Major Modules and Components

- Data loading, inspection and duplicate removal.
- Exploratory data analysis (class balance, message length, frequent words, word clouds).
- Text preprocessing function `clean_text()`.
- Train-test split and TF-IDF vectorization.
- Model definition and training loop for the four classifiers.
- Evaluation module (metrics table, confusion matrices, ROC and precision-recall curves, classification report).
- Interpretation module (Logistic Regression coefficients).
- Prediction function `predict_message()` for new messages.

9.3 Application or User Interface

We did not develop a separate application or graphical user interface. The system runs in a Google Colab notebook, where new messages are classified by calling the `predict_message()` function. The submitted project does not include deployment as a real-time mobile or messaging service.

9.4 Working System: Testing on New Messages

After training and evaluation, we use a `predict_message` function for new inputs. It applies the same preprocessing used during training, transforms the message with the already-fitted TF-IDF vectorizer, predicts the class using the selected model and returns the estimated spam probability.

```
def predict_message(text, model_name=None):
    model_name = model_name or best_model_name
    cleaned = clean_text(text)
    vec = tfidf.transform([cleaned])
    pred = fitted_models[model_name].predict(vec)[0]
    proba = fitted_models[model_name].predict_proba(vec)[0][1]
    label = 'SPAM' if pred == 1 else 'HAM'
```

Table 9. Predictions on manually supplied new messages.

Test message	Prediction	Reported spam probability
Congratulations! You have WON a FREE prize. Click here to claim now: bit.ly/claim123	SPAM	0.99
Hey, are we still meeting for lunch tomorrow at 1pm?	HAM	0.00
URGENT! You have won a 1 week FREE membership in our £100,000 Prize Jackpot! Txt WORD to 81010	SPAM	0.99

```
Message: Congratulations! You have WON a FREE prize. Click here to claim now: bit.ly/claim123
Cleaned: congratulation free prize click claim bit ly claim
Prediction: SPAM (spam probability: 0.99)

Message: Hey, are we still meeting for lunch tomorrow at 1pm?
Cleaned: hey still meeting lunch tomorrow pm
Prediction: HAM (spam probability: 0.00)

Message: URGENT! You have won a 1 week FREE membership in our £100,000 Prize Jackpot! Txt WORD to 81010
Cleaned: urgent week free membership prize jackpot txt word
Prediction: SPAM (spam probability: 0.99)
```

Figure 6. Output of the new-message test in the implementation.

The first message is classified as spam after preprocessing because its representation contains terms such as congratulation, free, prize, click and claim. The second message is classified as ham and is reduced to a more conversational representation. The third message is classified as spam and retains terms related to urgency, free membership, prizes and txt. These three examples are useful as a quick qualitative check, but they are not a substitute for the held-out test-set evaluation.

The results also give a small indication of how the model generalizes. A generic prize-scam message is correctly identified with a high reported spam probability. This example also shows why the choice of training dataset matters when the model is asked to handle new spam wording.

10. Results and Evaluation

10.1 Evaluation Metrics

Accuracy is the proportion of all test messages that are classified correctly. Precision tells us how many messages predicted as spam are actually spam, while recall tells us how many of the real spam messages are detected. F1-score combines precision and recall into a single measure, and ROC-AUC shows how well the model separates the two classes across different thresholds. Since spam is the minority class, precision and recall are especially useful; a model can have high accuracy while still allowing many spam messages through if its recall is poor.

10.2 Model Performance Comparison

Table 10. Model performance comparison on the test set (spam as the positive class).

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Linear SVM	0.982592	0.944882	0.916031	0.930233	0.993884
Random Forest	0.976789	0.981982	0.832061	0.900826	0.992442
Logistic Regression	0.970986	0.863309	0.916031	0.888889	0.991534
Multinomial Naive Bayes	0.974855	1.000000	0.801527	0.889831	0.986115

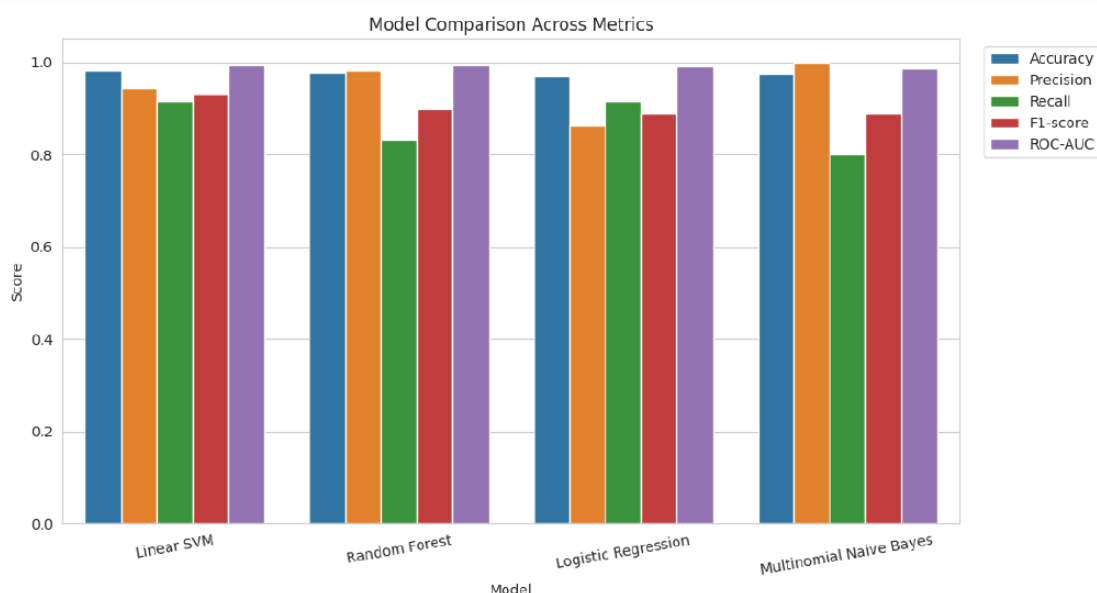


Figure 7. Model comparison across accuracy, precision, recall, F1-score and ROC-AUC.

Linear SVM records the highest accuracy, F1-score and ROC-AUC among the four models in this experiment. Its ROC-AUC of 0.993884 is also the highest. Random Forest has the highest precision among the models that do not reach 1.000000, at 0.981982, but its recall is lower at 0.832061. Multinomial Naive Bayes reaches a precision of 1.000000, but its recall is only 0.801527, meaning that the spam messages it flags are very precise while a larger number of actual spam messages are missed. Logistic Regression has the same recall as

Linear SVM, 0.916031, but lower precision and F1-score. Based on the project's ROC-AUC comparison, Linear SVM is therefore selected as the best-performing model.

10.3 Confusion Matrices

A confusion matrix breaks the predictions into true negatives, false positives, false negatives and true positives. Figure 8 presents the confusion matrices for all four classifiers so that their types of errors can be compared directly.

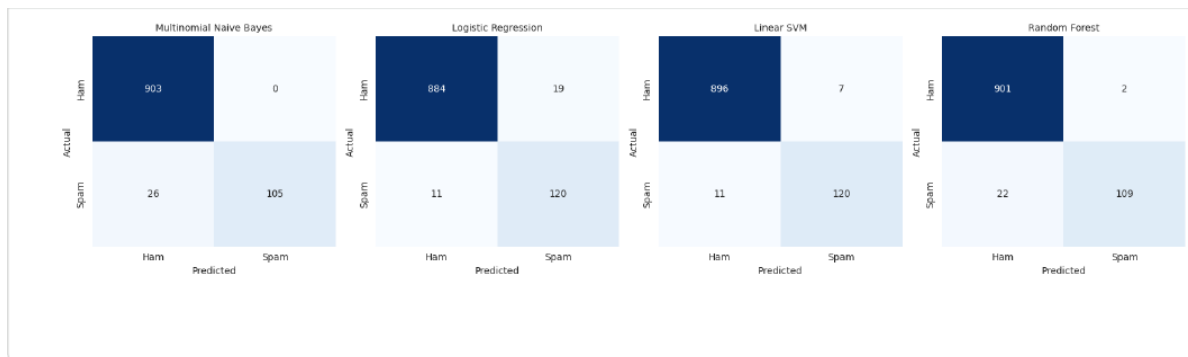


Figure 8. Confusion matrices for the four classifiers (Naive Bayes, Logistic Regression, Linear SVM, Random Forest).

For Linear SVM, 896 ham messages are correctly classified as ham, while 7 ham messages are incorrectly classified as spam. Of the spam messages, 11 are classified as ham and 120 are correctly identified as spam. These values are consistent with the precision, recall and F1-score reported for the best model.

10.4 ROC Curves

The ROC curves stay close to the upper-left corner, which indicates that all four classifiers separate the two classes well. Their ROC-AUC values are all high, ranging from 0.986 to 0.994, with Linear SVM achieving the highest value.

10.5 Precision-Recall Curves

Precision-Recall curves are especially useful for an imbalanced classification problem because they show the trade-off between finding positive cases and avoiding false positives. The average precision (AP) is 0.97 for Linear SVM and Random Forest, and 0.95 for Logistic Regression and Multinomial Naive Bayes. Across much of the useful recall range, the curves maintain high precision.

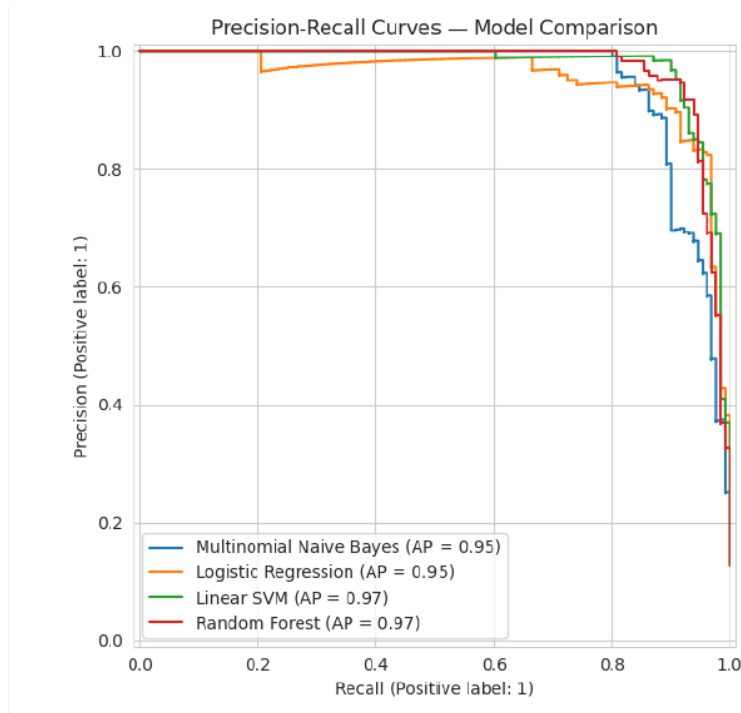


Figure 9. Precision-Recall curves for model comparison.

10.6 Detailed Classification Report: Linear SVM

Table 11. Detailed classification report for Linear SVM.

Class	Precision	Recall	F1-score	Support
Ham	0.99	0.99	0.99	903
Spam	0.94	0.92	0.93	131
Accuracy			0.98	1,034
Macro average	0.97	0.95	0.96	1,034
Weighted average	0.98	0.98	0.98	1,034

10.7 Feature Interpretation: Most Influential Words

The project uses Logistic Regression coefficients to identify features associated with the two classes. Positive coefficients push the model toward the spam class, while negative coefficients push it toward ham. Terms such as txt, call, mobile, free, reply, text, claim, prize, stop, service, tone, win, chat, new, cash, uk, per, com, video, and ringtone are among the stronger spam-oriented features. On the ham side, terms such as ok, gt, lt, home, later, da, lor, got, way, oh, still, say, going, much, lol, hey, sorry, good, and wat appear among the stronger ham-oriented features.

We use the Logistic Regression coefficients to see which features are associated more strongly with each class. Positive coefficients move the prediction toward spam, while negative coefficients move it toward ham. Some of the stronger spam-oriented features include txt, call, mobile, free, reply, text, claim, prize, stop, service, tone, win, chat, new, cash, uk, per, com, video and ringtone. On the ham side, terms such as ok, gt, lt, home, later,

da, lor, got, way, oh, still, say, going, much, lol, hey, sorry, good and wat are among the stronger features.

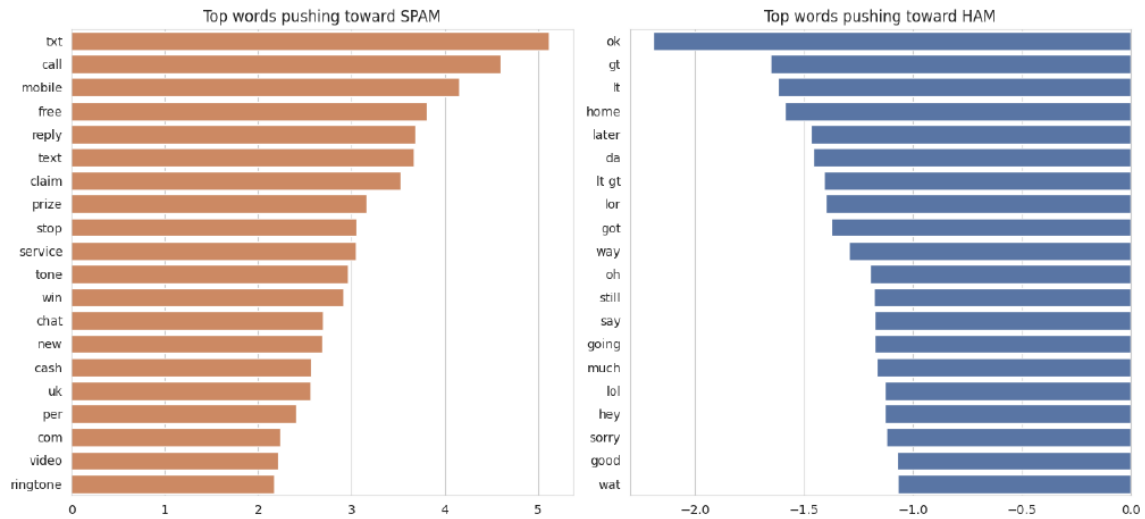


Figure 10. Most influential Logistic Regression features for spam and ham.

These words should not be treated as fixed spam or ham rules. The classifier looks at the combined TF-IDF representation of the entire message. For example, seeing the word free or prize alone is not enough to decide that a message is spam.

10.8 Discussion

- **Data quality and leakage prevention:** removing the 403 duplicate messages before the split reduces the chance that identical messages appear in both training and testing, which could otherwise make the evaluation look better than it really is.
- **Effect of class imbalance:** because the class ratio is about 6.9:1, accuracy by itself is not enough to judge the spam detector. Using class weighting for three models and examining precision, recall, F1-score and ROC-AUC gives a more useful picture of performance.
- **Model selection:** Linear SVM is selected as the best model based on ROC-AUC. Its confusion matrix contains only seven false-positive ham messages and eleven spam messages that were missed.
- **Practical meaning:** the results show that traditional machine learning can perform strongly on this dataset when the text is prepared carefully. At the same time, very high precision does not automatically mean that most spam is detected, and high accuracy can be misleading when the positive class is relatively small.

11. Error Analysis / NLP Failure Analysis

The notebook presents aggregate errors through the confusion matrices rather than listing individual misclassified messages. Therefore, the analysis below combines the error counts with the specific failure cases that can be seen in the preprocessing and prediction outputs.

11.1 Error Summary by Model

Table 12. Error counts from the confusion matrices and derived error rates (test set: 903 ham, 131 spam).

Model	False positives (ham flagged as spam)	False negatives (spam missed)	Total errors	Ham wrongly blocked (of 903)	Spam missed (of 131)
Linear SVM	7	11	18	0.78%	8.40%
Random Forest	2	22	24	0.22%	16.79%
Multinomial Naive Bayes	0	26	26	0.00%	19.85%
Logistic Regression	19	11	30	2.10%	8.40%

Linear SVM makes the fewest mistakes overall, with 18 errors out of 1,034 test messages. Logistic Regression produces the highest number of false positives, incorrectly flagging 19 legitimate messages, which is consistent with its lower precision of 0.863. Naive Bayes and Random Forest are more conservative: they rarely mark legitimate messages as spam, but they miss 26 and 22 of the 131 spam messages respectively. These two types of errors have different effects: a false positive can hide a legitimate message, while a false negative allows a spam message to pass through.

11.2 Failure Cases Observed in the Pipeline

Table 13. Failure cases visible in the preprocessing and prediction outputs.

Case	Observed output	Issue and likely reason
Link without http/www	bit.ly/claim123 is cleaned to "bit ly claim"	The URL pattern only matches http or www, so the link is split into ordinary words and the link signal is lost.
Amount and shortcode	£100,000 and 81010 do not appear in "urgent week free membership prize jackpot txt word"	Uppercase placeholders (CURRENCY, NUM) are deleted by the next step, which keeps only lowercase letters.
Numbers inside words	"1pm" becomes "pm"	Digits are removed, so time and quantity information is lost.
Empty messages	12 messages contain no tokens after cleaning	Messages made only of stopwords, numbers, symbols or single characters carry no features, so the classifier has nothing to learn from.

11.3 Discussion of Failure Types

- **Informal language and spelling variation:** SMS messages contain forms such as wif, oni, dun, hor, lor and wat. The standard English stopword list and lemmatizer do not convert these forms into standard English words. Some of these tokens are still useful for the classifier, such as lor, da and wat as ham indicators, but terms that occur in fewer than two training messages are removed because of `min_df=2`.
- **Context dependence and ambiguity:** words such as call, get and ur appear frequently in both classes. Since the bag-of-words representation captures word order only through bigrams, a spam message that avoids typical promotional terms can look similar to a ham message. This may help explain some of the spam messages that were missed.
- **Encoding artifacts:** tokens such as gt and lt appear among the stronger ham features. They seem to come from encoded characters in the original text rather than from meaningful words, which means the model may be using some formatting noise as a signal.
- **Information lost during preprocessing:** as shown in Table 13, the intended link, currency and number information is not retained in the final cleaned text, even though these details can be useful clues in spam messages.

12. Ethical Considerations and Limitations

12.1 Ethical Considerations

- **Privacy:** the dataset contains real SMS messages, which are personal communications. If a similar filter were deployed, it would also process private messages. Appropriate consent, security measures and minimal data retention would therefore be important.
- **Harm from false positives:** marking a genuine message as spam can cause a user to miss important information. The best model produces 7 false positives, while Logistic Regression produces 19, showing that this is a real practical concern. The decision threshold should therefore be chosen according to how the system will be used.
- **Dataset representativeness and bias:** the collection consists of UK English messages from the mid-2000s. Slang, regions, languages and spam techniques outside this dataset are not represented, so performance may be different for other groups of users or newer messaging environments.

12.2 Limitations

- The project relies on TF-IDF and traditional machine learning instead of newer transformer-based language models.
- The dataset contains UK English SMS messages from the mid-2000s, so it may not fully represent current slang, vocabulary or modern spam tactics.
- The classes are imbalanced, which means measures such as precision, recall and F1-score provide more useful information than accuracy alone.
- The manually supplied new-message examples are only a small sanity check and should not be treated as a statistically significant external validation set.
- The model is trained for the ham-versus-spam task represented by this dataset, so its performance may not transfer equally well to other languages, regions, communication channels or modern messaging environments.
- The results are based on a single train-test split. No cross-validation or hyperparameter tuning was reported, so small differences between the models should be interpreted carefully, especially because the test set contains only 131 spam messages.
- The URL, currency and number placeholders are removed during the cleaning process described in Section 5, so those signals are not ultimately used by the models.
- The project does not include deployment as a real-time mobile or messaging service. Since spam vocabulary changes over time, a production system would also need regular retraining using newer data.

13. Conclusion

This project developed a complete Natural Language Processing and machine learning workflow for detecting SMS spam using the UCI SMS Spam Collection. The process started with 5,572 messages, and 403 duplicates were removed before model development, leaving 5,169 unique records. The exploratory analysis also showed a clear class imbalance, with about 87.4% ham and 12.6% spam messages.

During preprocessing, URLs, currency symbols and long numbers are normalized, punctuation is removed, the messages are tokenized, stopwords are removed and lemmatization is applied. TF-IDF then converts the cleaned text into 3,000 features using unigrams and bigrams. Four traditional classifiers are trained and compared using several evaluation measures.

Among the models tested, Linear SVM gives the strongest overall result according to the ROC-AUC comparison. It achieves 98.26% accuracy, 94.49% precision, 91.60% recall, 93.02% F1-score and 0.993884 ROC-AUC. The confusion matrix and classification report also show strong performance for both ham and spam. The three new-message examples were classified as expected. Overall, the stated objectives were met: the dataset was prepared, a complete preprocessing and TF-IDF pipeline was developed, four models were trained and compared, and the final pipeline was tested on new messages.

One of the main lessons from the project is that careful data handling can be just as important as model choice. Removing duplicates before the split, fitting TF-IDF only on training data and accounting for class imbalance all help make the evaluation more reliable. At the same time, the age of the dataset and the way spam language changes over time show why fresh data, regular retraining and potentially more advanced language models would be useful in future work.

14. Future Scope

The following points were not implemented in the submitted pipeline. They are possible improvements that could be explored in a future version of the project.

- Improve the handling of placeholder tokens, for example by using lowercase placeholders, and recognize links such as bit.ly so that link, amount and shortcode information is retained.
- Display and analyse the misclassified messages, and use cross-validation and hyperparameter tuning to make the model comparison more reliable.
- Explore word-embedding methods such as Word2Vec or GloVe to capture relationships between words.

- Evaluate transformer-based models such as BERT to obtain richer contextual representations.
- Use a larger and more recent SMS dataset to capture current spam language and tactics, with periodic retraining for a production setting.
- Use multilingual datasets to build classifiers that can handle messages written in different languages.
- Experiment with threshold tuning so that the balance between false positives and false negatives can be adjusted for the intended application.
- Integrate the trained model into a real-time message-screening application or API.

15. References

- [1] Almeida, T. A., Gomez Hidalgo, J. M., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. Proceedings of the 11th ACM Symposium on Document Engineering (DocEng '11), 259-262.
- [2] UCI Machine Learning Repository. (n.d.). SMS Spam Collection dataset. <https://archive.ics.uci.edu/dataset/228/sms+spam+collection>
- [3] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830. (scikit-learn documentation: <https://scikit-learn.org>)
- [4] Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media. (NLTK documentation: <https://www.nltk.org>)
- [5] pandas, NumPy, Matplotlib, Seaborn and WordCloud library documentation, as used in the Python/Google Colab implementation.

16. GitHub Repository

The complete implementation, including the Google Colab notebook, dataset reference and reproduction instructions, is maintained in the project repository rather than being included in full in this report.

- GitHub Repository Link: <https://github.com/Sarvesh-Wani-07/spam-detection-NLP>

17. Team Contribution

The specific contribution of each team member is provided below.

Table 14. *Contribution of each team member.*

Team Member	PRN	Specific Contribution
Sarvesh Wani	25030422140	PowerPoint presentation and Report
Sanando Maitra	25030422105	PowerPoint presentation and Report
Priyanshu Akshat	25030422095	Coding And Audit Sheet

18. Annexure

Annexure A: Reproducibility Notes

- Random seed: `RANDOM_STATE = 42` is used for the train-test split and for all models that use randomness.
- NLTK resources required: stopwords, punkt, punkt_tab, wordnet and omw-1.4.
- Dataset file: SMSSpamCollection (tab-separated, no header row), loaded using pandas.
- Environment: Google Colab with scikit-learn, NLTK, pandas, NumPy, Matplotlib, Seaborn and WordCloud.

Annexure B: Evaluation Code Snippet

```
results.append({
    'Model': name,
    'Accuracy': accuracy_score(y_test, y_pred),
    'Precision': precision_score(y_test, y_pred),
    'Recall': recall_score(y_test, y_pred),
    'F1-score': f1_score(y_test, y_pred),
    'ROC-AUC': roc_auc_score(y_test, y_proba),
})
```

The full source code is available in the GitHub repository (Section 16).